

UNIVERSIDAD AMERICANA
Facultad de Ingeniería y Arquitectura



Programación Orientada a Objetos II

Pruebas Unitarias Básicas: App de Tareas

Estudiante:

- Sara Alejandra Zambrana Taylor

Docente:

- José Durán García

Managua, Nicaragua

(09/06/2026)

I. Introducción

En esta práctica se desarrolló una aplicación de gestión de tareas utilizando Android Studio y Jetpack Compose. El objetivo principal fue implementar pruebas unitarias y pruebas de interfaz para validar el correcto funcionamiento de la lógica de negocio y de los componentes visuales de la aplicación.

Además, se implementaron mejoras adicionales como filtros de tareas, ordenamiento alfabético y cálculo del porcentaje de tareas completadas.

I. I Descripción de la app

La aplicación permite:

- Agregar tareas.
- Eliminar tareas.
- Marcar tareas como completadas.
- Mostrar cantidad de tareas pendientes.
- Filtrar tareas completadas.
- Filtrar tareas pendientes.
- Ordenar tareas alfabéticamente.
- Mostrar porcentaje de tareas completadas.

I. II Pruebas Unitarias Implementadas

Se desarrollaron pruebas unitarias para validar la lógica de negocio del repositorio de tareas.

Casos evaluados:

- Agregar tarea.
- Eliminar tarea.
- Marcar tarea como completada.
- Contar tareas pendientes.
- Lista vacía.
- Obtener tareas completadas.
- Ordenar tareas alfabéticamente.
- Calcular porcentaje de tareas completadas.
- Calcular porcentaje cuando no existen tareas.

Resultado: Todas las pruebas unitarias se ejecutaron exitosamente.

I. III Pruebas de Interfaz Implementadas

Se desarrollaron pruebas de interfaz utilizando Jetpack Compose Testing.

Casos evaluados:

- Agregar tarea.
- Verificar funcionamiento del botón Agregar.
- Eliminar tarea.

- Mostrar cantidad de pendientes.
- Verificar entrada de texto.
- Prueba negativa intencional.

Resultado: Las pruebas de interfaz validan correctamente el comportamiento visual y funcional de la aplicación.

I. IV Prueba Negativa

Se diseñó una prueba unitaria que verifica el conteo de tareas pendientes. Sin embargo, la prueba fue creada intencionalmente con una expectativa incorrecta para provocar un fallo y comprobar que el sistema de pruebas detecta errores adecuadamente.

¿Por qué falla la prueba?

La prueba falla porque se agregan únicamente dos tareas pendientes al repositorio, pero la aserción espera que existan tres. El método contarTareasPendientes() funciona correctamente y retorna el valor real de tareas pendientes almacenadas.

Por lo tanto:

- Valor esperado por la prueba: 3
- Valor obtenido realmente: 2

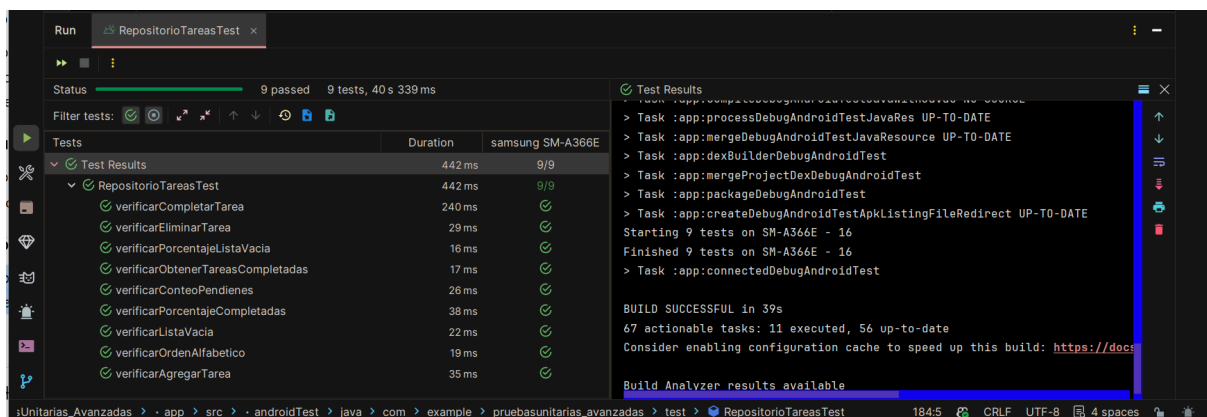
Debido a esta diferencia, la prueba falla.

¿Cómo corregir el error?

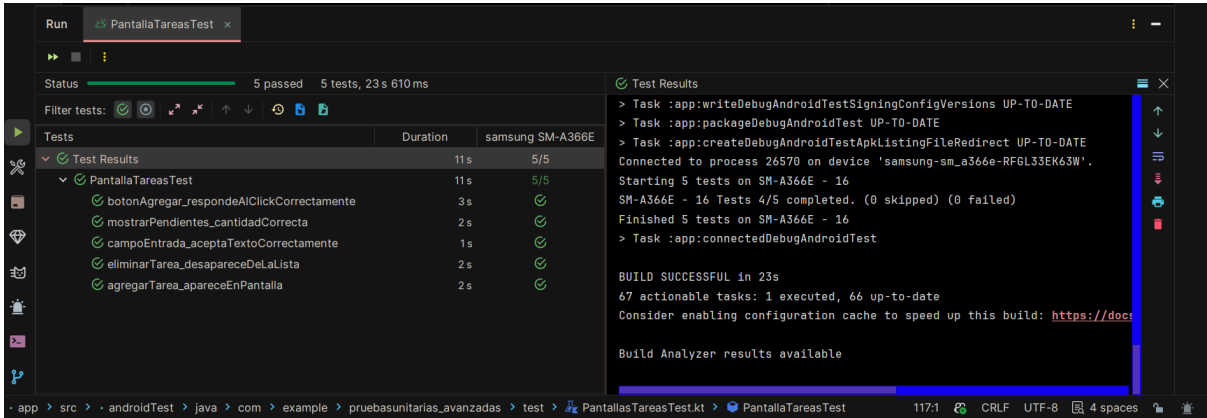
Se puede corregir modificando la aserción para que espere el valor correcto (2) o agregando una tercera tarea pendiente antes de realizar la verificación.

II. Capturas de Evidencia

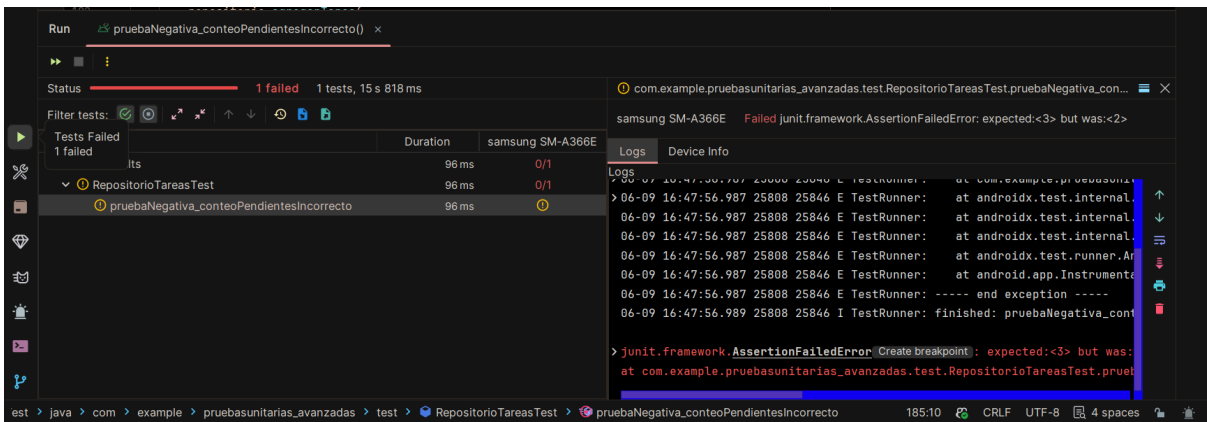
- Ejecución de pruebas unitarias



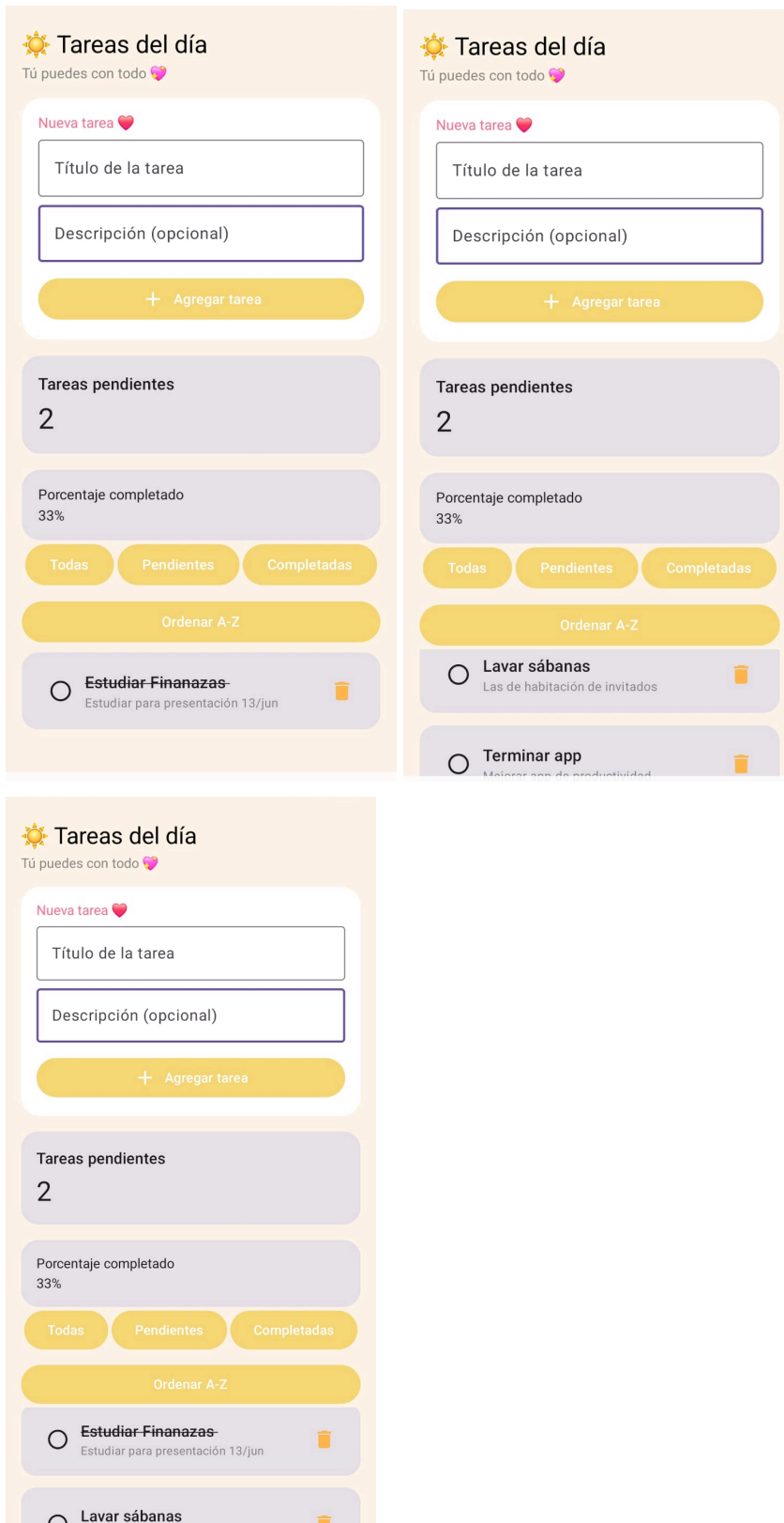
- **Ejecución de pruebas de interfaz**



- **Ejecución de prueba negativa**



- **Funcionamiento de la aplicación**



III. Preguntas de Análisis

- **¿Qué válida la prueba unitaria?**

La prueba unitaria valida que la lógica de negocio de la aplicación funcione correctamente. En este proyecto se verificó que el repositorio de tareas agregue, elimine y complete tareas correctamente, además de calcular la cantidad de tareas pendientes, filtrar tareas y obtener el porcentaje de tareas completadas.

- **¿Qué válida la prueba de interfaz?**

La prueba de interfaz valida que los elementos visuales respondan correctamente a las acciones del usuario. En este proyecto se comprobó que el usuario pueda escribir una tarea, agregarla mediante el botón correspondiente, visualizarla en la lista, eliminarla y observar correctamente el contador de tareas pendientes.

- **Diferencias entre ambos tipos de pruebas**

Las pruebas unitarias se enfocan en comprobar la lógica interna de la aplicación, sin interactuar con la interfaz gráfica. Por ejemplo, verifican que el repositorio cuente correctamente las tareas pendientes o las ordene alfabéticamente. En cambio, las pruebas de interfaz simulan acciones reales del usuario sobre la pantalla, como escribir en un campo de texto o presionar botones, para verificar que la aplicación se comporte correctamente desde la perspectiva del usuario.